

Types of Linked List:

Now we have an idea about Linked List and how a linked list works. One important thing to remember is that everything regarding the linked list is created and maintained manually by us. We have designed the node class and also created the linked list. After that, we implemented the idea of indexing along with different operations such as insert, removal, rotation, and so on. If you think about it all these operations are a bit complicated because of our node class design which only has the option of moving from one node to another in a forward manner.

This brings the question can we design our linked list in such a way that we can traverse from one node to another in a forward and backward manner? In addition, can we make our linked list circular and is there any way where we will not need to handle the head explicitly?

To answer all these questions, the types of the linked list have been introduced. Linked Lists type can be determined by considering three different categories. Each category is independent of the other two options. We need to understand first what these options mean.

Category 1	Category 2	Category 3
Non-Dummy Headed	Singly	Linear
Dummy Headed	Doubly	Circular

Non-Dummy Headed: It means that the linked list's first node (the head) contains an item along with the information of the next node. For example, the lists we have used till now are Non-Dummy headed.

Example: $10 \rightarrow 20 \rightarrow 30 \rightarrow 40$. In this Linked list, the first node stores the value head.

Dummy-Headed: This refers to the linked list where a head node is a node-type object but it does not store any element on its own. Rather it stores the information of the next node where the starting value is stored. The benefit of this is that we do not need to handle the first item of the data carefully now. To illustrate, we can remove or add items at beginning of the list without handling the head delicately. The reason is the first item is stored after the dummy head.

Example: $DH \rightarrow 10 \rightarrow 20 \rightarrow 30 \rightarrow 40$. In this linked list, DH refers dummy head which is a node without any element. The first item is stored after the dummy head.

Singly: It means every node has only the information of its next node. The reason is the design of the Node class has only one variable (next variable). Up until now all the list we worked on is Singly Linked List.

10 → 20 → 30 → 40

Doubly: It means a node knows its next node and its previous node's information. To achieve this we need to modify the Node class design where we store the previous node's information. Example of Doubly Node class in python

10 ⇌ 20 ⇌ 30 ⇌ 40

```
class DoublyNode:
```

```
    def __init__(self, elem, next, prev):
```

```
        self.elem = elem
```

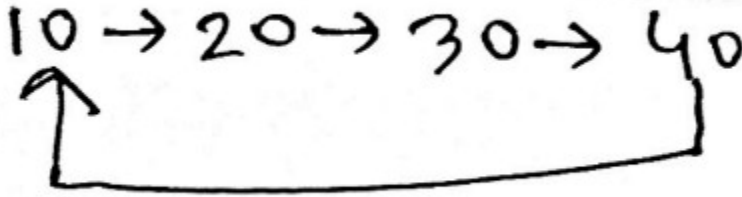
```
        self.next = next # To store the next node's reference.
```

```
        self.prev = prev # To store the previous node's reference.
```

Linear: Linear linked list refers to the linked list where the linked list is linear in structure. To illustrate, the last node of the linked list will refer to None.

10 → 20 → 30 → 40

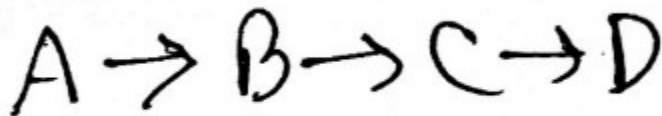
Circular: Circular linked list refers to a linked list where the linked list is circular in structure. To illustrate, the last node of the linked list will refer to the starting node.



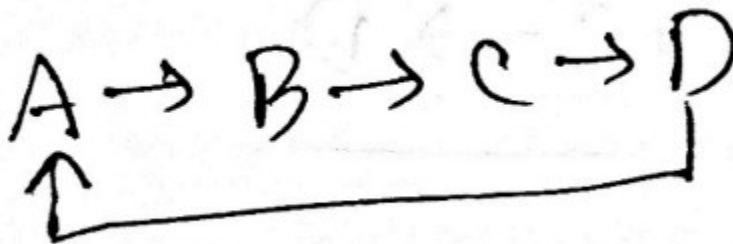
Now that we understand the options of each category, we can easily identify the type of a linked list. The main idea is that a linked list will contain one option from each category mentioned above. The option of one category does not have any impact on other categories.

The linked list we are taught until now is Non-Dummy Headed Singly Linear Linked List. You can think of this as a default linked list. It means if not mentioned, think of the head as Non-Dummy Head, Connection as Singly, and the structure as Linear. Now combining the categories the linked list can be 8 types. DH means dummy head in below diagram

1. Non-Dummy Headed Singly Linear Linked List:



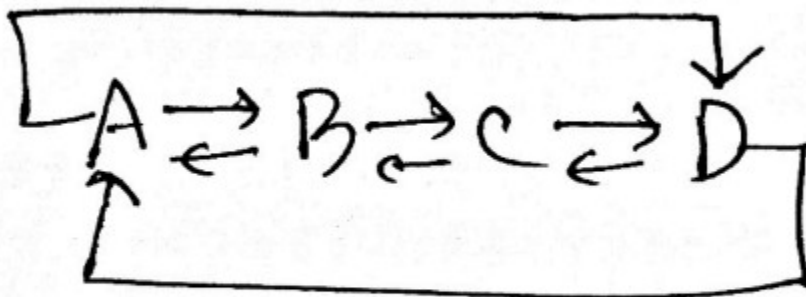
2. Non-Dummy Headed Singly Circular Linked List:



3. Non-Dummy Headed Doubly Linear Linked List:

$A \rightleftarrows B \rightleftarrows C \rightleftarrows D$

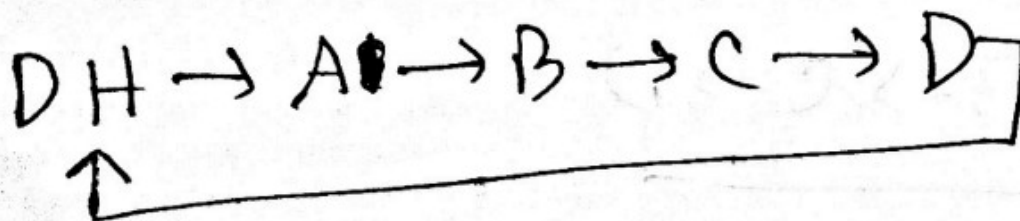
4. Non-Dummy Headed Doubly Circular Linked List:



5. Dummy Headed Singly Linear Linked List:

DH $\longrightarrow$ A $\longrightarrow$ B $\longrightarrow$ C $\longrightarrow$ D

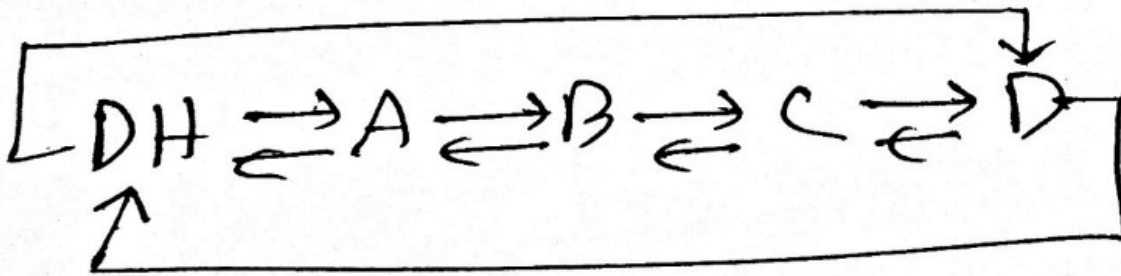
6. Dummy Headed Singly Circular Linked List:



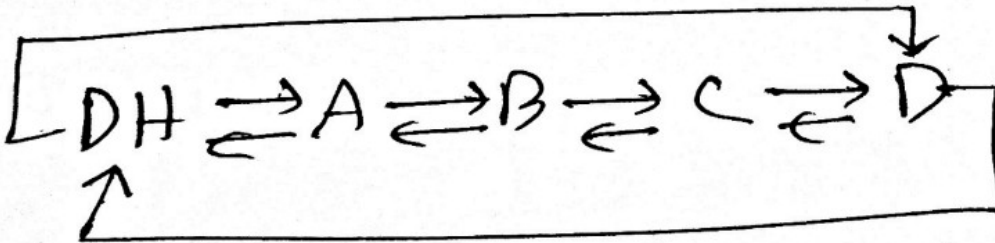
7. Dummy Headed Doubly Linear Linked List:

DH $\rightleftarrows$ A $\rightleftarrows$ B $\rightleftarrows$ C $\rightleftarrows$ D

8. Dummy Headed Doubly Circular Linked List:



Dummy Headed Doubly Circular Linked List Operations:



To practice we will learn about the dummy-headed doubly circular linked list. As the title suggests, the list will have a dummy head, doubly connection, and circular structure. For this reason, the Node class will be

```
class DoublyNode:
    def __init__(self, elem, next, prev):
        self.elem = elem
        self.next = next # To store the next node's reference.
        self.prev = prev # To store the previous node's reference.
```

Creation:

It is similar to creating a linked list. The difference is that we need to ensure the connections in a proper way.

The below function takes an array and creates a dummy-headed doubly circular linked list.

```
1 def createList(a):
2     dh = DoublyNode(None, None, None)
3     dh.next = dh
4     dh.prev = dh
5     tail = dh
6
7     for i in range(len(a)):
8         n = DoublyNode(a[i], dh, tail)
9         tail.next = n
10        tail = tail.next
11        dh.prev = tail
12
13    return dh
```

Iteration:

It is similar to the previous linked list. The difference is that it will start from the next item of the dummy head and run till the pointer comes back to the dummy head.

```
1 def iteration(dh):
2     temp = dh.next
3     while temp != dh:
4         print(temp.elem)
5         temp = temp.next
```

NodeAt:

The nodeAt method is similar to the previous one. The difference is that the dummy head does not represent any index as it does not have any value.

```

1 def nodeAt(dh, idx):
2     temp = dh.next
3     c = 0
4     while temp != dh:
5         if c == idx:
6             return temp
7         c += 1
8         temp = temp.next
9     return None # Invalid Index

```

Note that, if you want to count the number of total nodes, you should ignore the dummy head too.

Insertion:

To insert a new node in the list, you need the reference to the predecessor to link in the new node. Unlike for a singly-linked linear list, there is no "special" case here, since there is always a valid predecessor node available, thanks to the dummy head.

```

1 def insertion(dh, elem, idx):
2     # Assuming the idx is valid
3     node_to_insert = DoublyNode(elem, None, None)
4     indexed_node = nodeAt(dh, idx) # Retriving the node at that index
5     prev_node = indexed_node.prev # There will always be a previous node
6     # Change the connection
7     # Observe that no special case is needed
8     node_to_insert.next = indexed_node
9     node_to_insert.prev = prev_node
10    prev_node.next = node_to_insert
11    indexed_node.prev = node_to_insert

```

Removal:

Removing an element from the list is done by removing the node that contains the element. Just like inserting a new node in a list, removing requires that you have the reference to the predecessor node. Since we're using a doubly-linked list, finding a predecessor of a node is trivial — it's `n.prev`. And thanks to the dummy head, there is no "special" case here as well.

```
1 def removal(dh, idx):
2     # Assuming the idx is valid
3     node_to_remove = nodeAt(dh, idx)
4     prev_node = node_to_remove.prev
5     next_node = node_to_remove.next
6     # Change the connection
7     # No special case is needed
8     prev_node.next = next_node
9     next_node.prev = prev_node
10    node_to_remove.next = None
11    node_to_remove.prev = None
12    return node_to_remove.elem # Returning the removed element
```